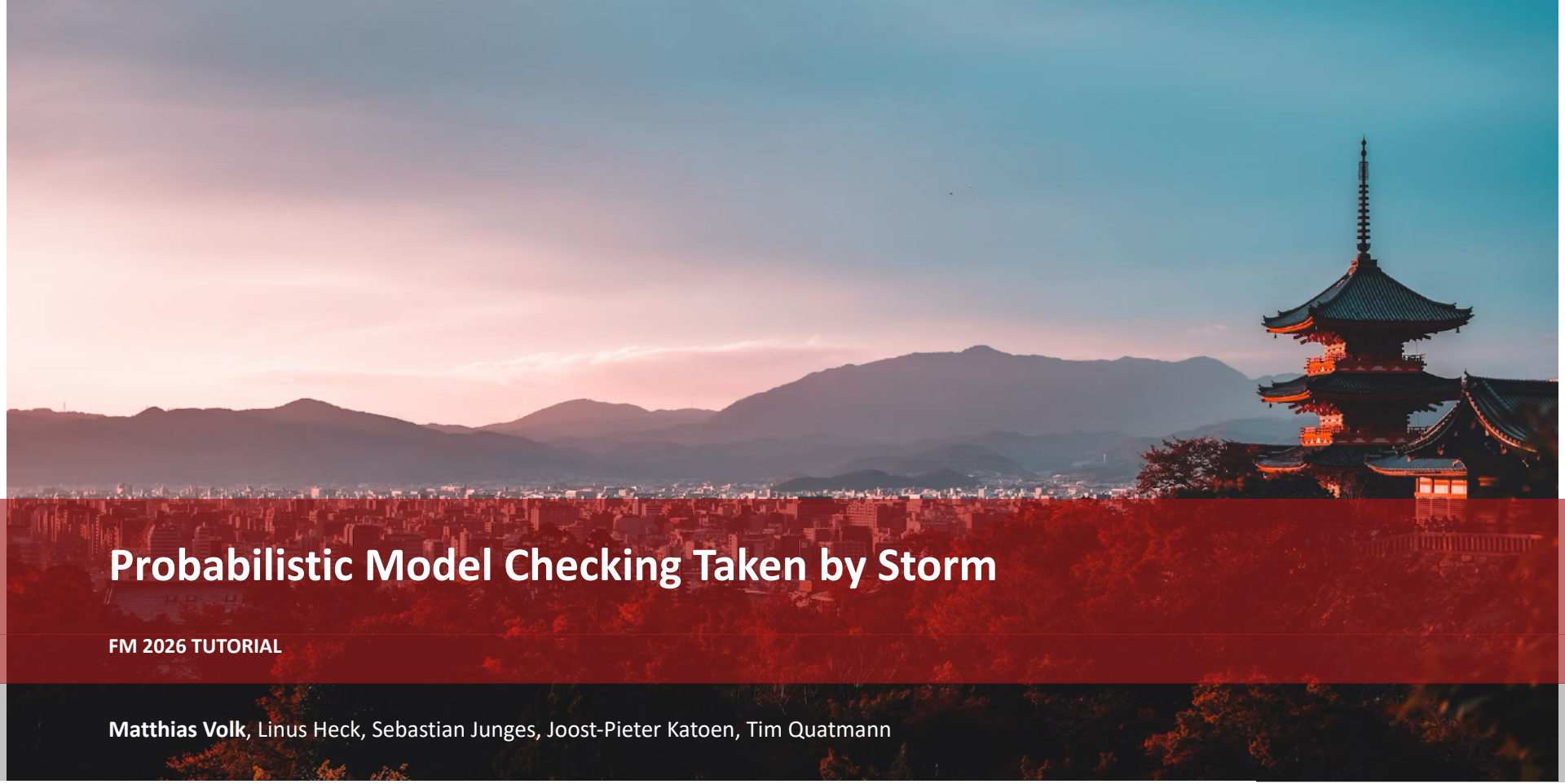


Probabilistic Model Checking Taken by Storm

FM 2026 TUTORIAL

Matthias Volk, Linus Heck, Sebastian Junges, Joost-Pieter Katoen, Tim Quatmann

Formal System Analysis Group

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

About me

volkm.github.io

- Assistant Professor in Formal System Analysis Group at Eindhoven University of Technology, NL
- interested in the improvement of safety-critical systems through formal methods

- focus on probabilistic model checking
- one of main developers of Storm model checker

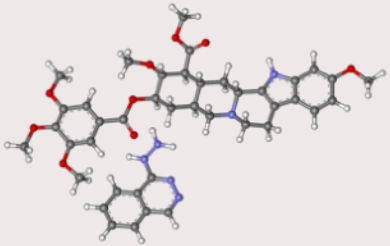
- research on (dynamic) fault trees
- co-developer of SAFEST tool

Probabilistic behavior is everywhere

Component failures

Randomized Algorithms

Artificial Intelligence



Biological / Chemical Systems

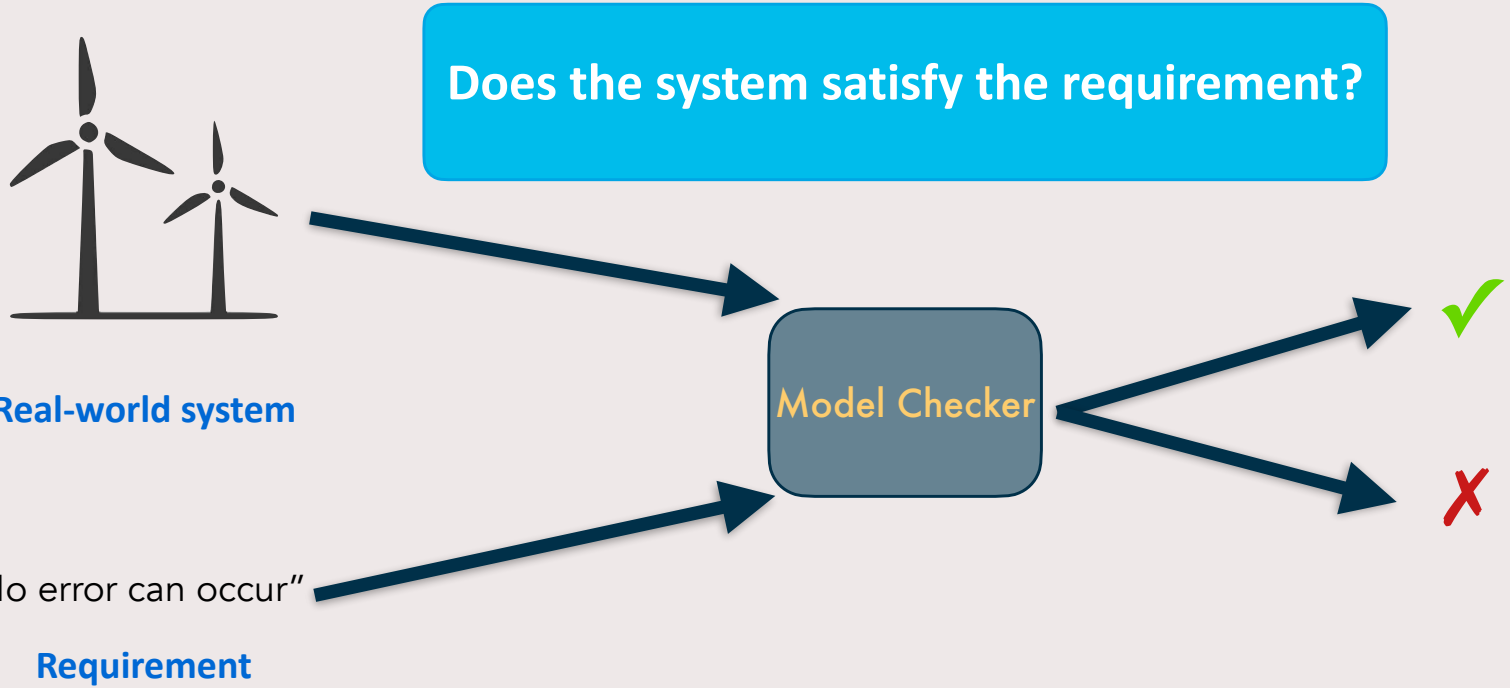
Arrivals / Delays

Board games

TU/e

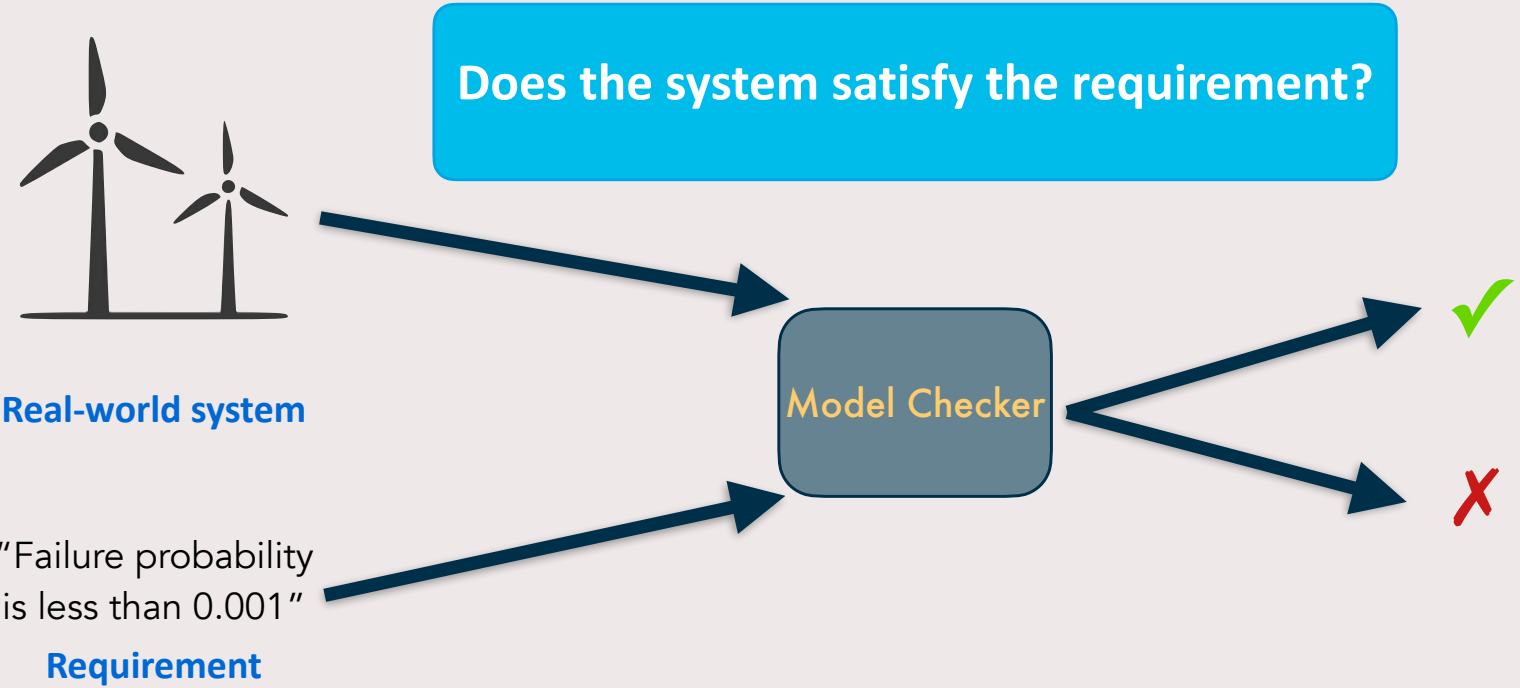
[Baier, Katoen, 2008]

Model checking



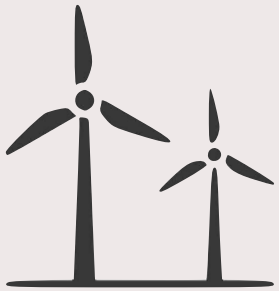
[Baier, Katoen, 2008]

Probabilistic model checking



[Baier, Katoen, 2008]

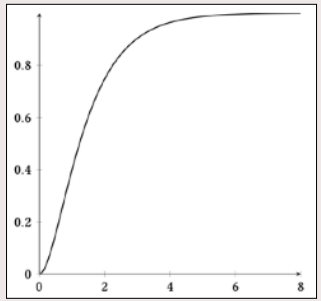
Probabilistic model checking



Real-world system

Does the system satisfy the requirement?

Model Checker



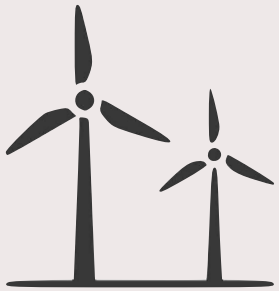
Analysis Result

"Failure probability within time t"

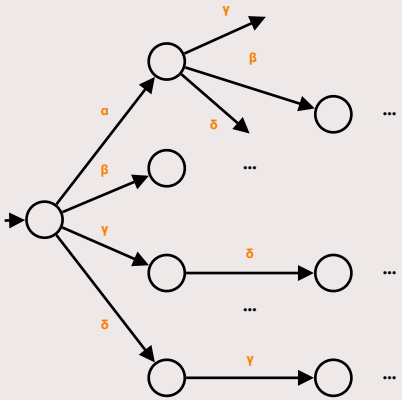
Requirement

TU/e

Probabilistic model checking



Real-world system



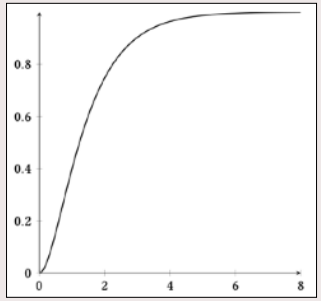
Markov model

Storm

PRISM model checker

Modest

Model Checker



Analysis Result

"Failure probability within time t"

Requirement

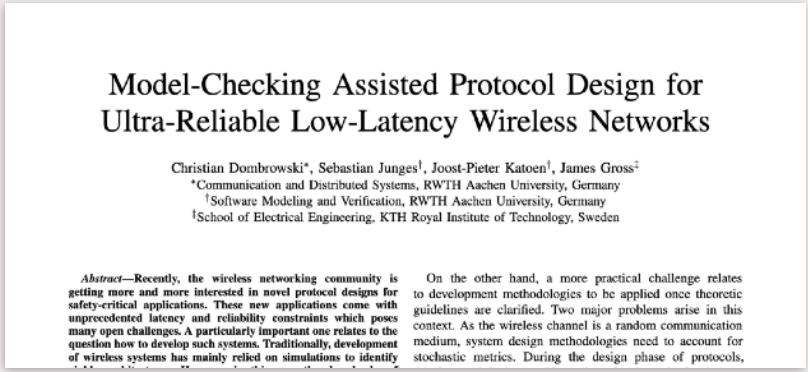
$\mathbf{P}_{=?}\left(\Diamond^{\leq t}\ \text{SysFail}\right)$

Formal Specification

TU/e

Applications of probabilistic model checking

- Network protocols

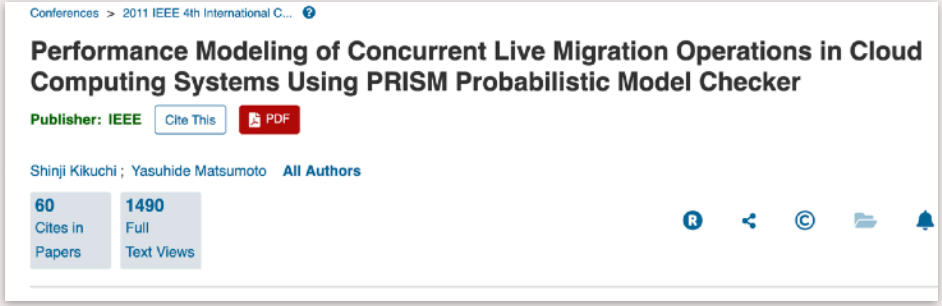


Model-Checking Assisted Protocol Design for Ultra-Reliable Low-Latency Wireless Networks

Christian Dombrowski*, Sebastian Junges†, Joost-Pieter Katoen†, James Gross‡
*Communication and Distributed Systems, RWTH Aachen University, Germany
†Software Modeling and Verification, RWTH Aachen University, Germany
‡School of Electrical Engineering, KTH Royal Institute of Technology, Sweden

***Abstract*—Recently, the wireless networking community is getting more and more interested in novel protocol designs for safety-critical applications. These new applications come with unprecedented latency and relibility constraints which poses many open challenges. A particularly important one relates to the question how to develop such systems. Traditionally, development of wireless systems has mainly relied on simulations to identify**

On the other hand, a more practical challenge relates to development methodologies to be applied once theoretic guidelines are clarified. Two major problems arise in this context. As the wireless channel is a random communication medium, system design methodologies need to account for stochastic metrics. During the design phase of protocols,



Conferences > 2011 IEEE 4th International C...

Performance Modeling of Concurrent Live Migration Operations in Cloud Computing Systems Using PRISM Probabilistic Model Checker

Publisher: IEEE | Cite This | PDF

Shinji Kikuchi ; Yasuhide Matsumoto **All Authors**

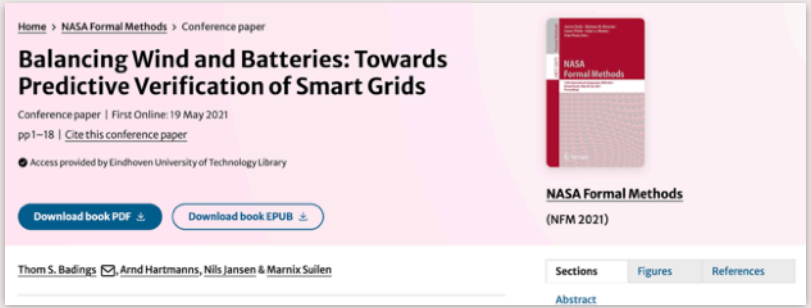
60 Cites in Papers

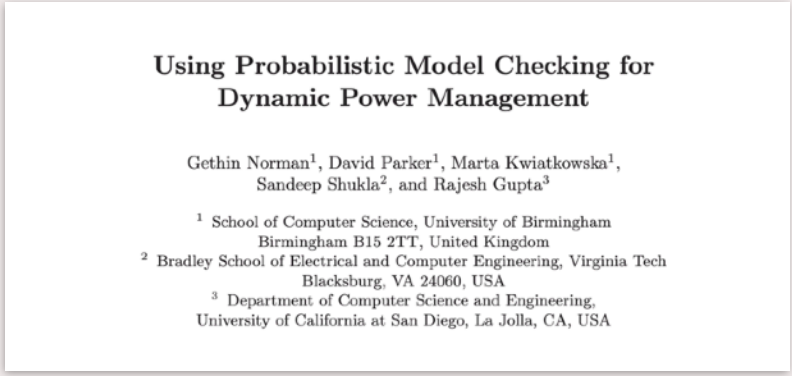
1490 Full Text Views

TU/e

Applications of probabilistic model checking

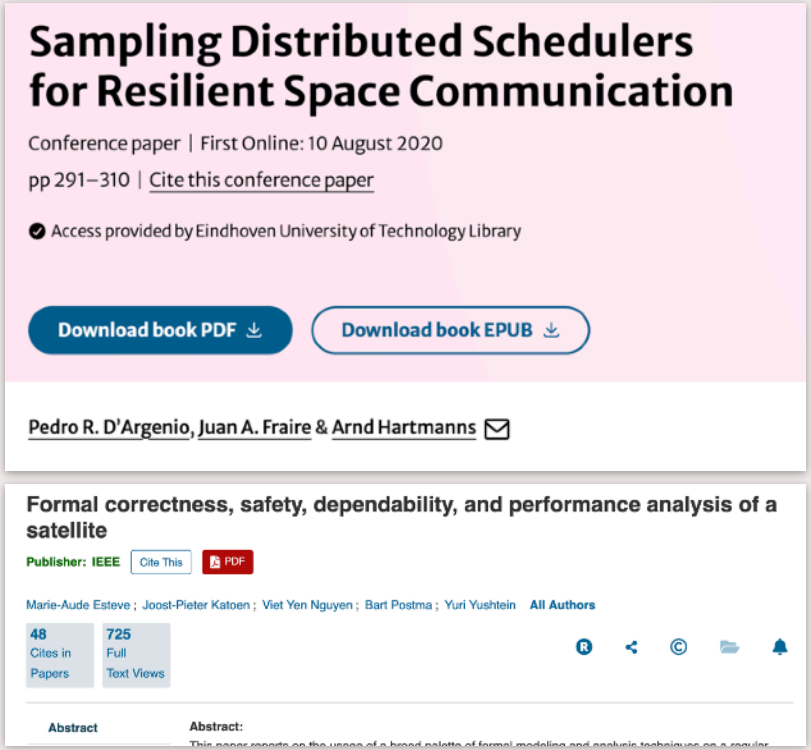
- Network protocols
- Power management





Applications of probabilistic model checking

- Network protocols
- Power management
- Reliability and Safety:
  - space



Sampling Distributed Schedulers for Resilient Space Communication

Conference paper | First Online: 10 August 2020

pp 291–310 | Cite this conference paper

Access provided by Eindhoven University of Technology Library

Download book PDF Download book EPUB

Pedro R. D'Argenio, Juan A. Fraire & Arnd Hartmanns

Formal correctness, safety, dependability, and performance analysis of a satellite

Publisher: IEEE Cite This PDF

Marie-Aude Esteve ; Joost-Pieter Katoen ; Viet Yen Nguyen ; Bart Postma ; Yuri Yushtein All Authors

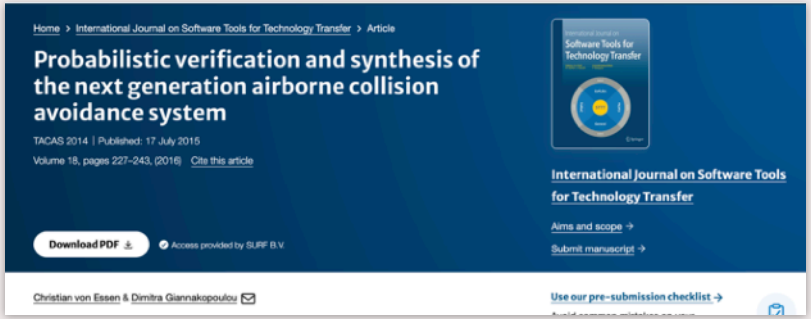
48 Cites in Papers 725 Full Text Views

Abstract

Abstract:

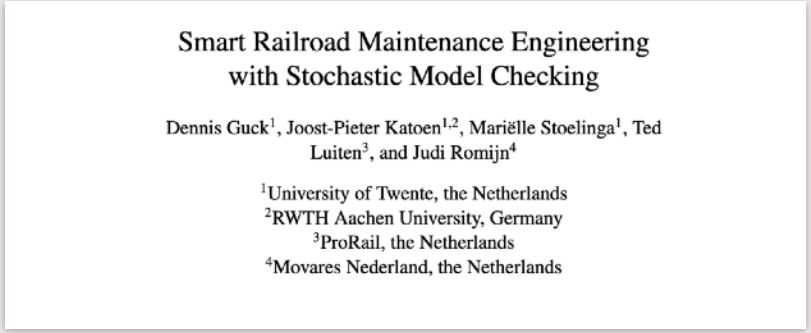
Applications of probabilistic model checking

- Network protocols
- Power management
- Reliability and Safety:
  - space
  - aerospace
  - automotive



Applications of probabilistic model checking

- Network protocols
- Power management
- Reliability and Safety:
  - space
  - aerospace
  - automotive
  - railway



Smart Railroad Maintenance Engineering with Stochastic Model Checking

Dennis Guck[1], Joost-Pieter Katoen[1,2], Mariëlle Stoelinga[1], Ted Luiten[3], and Judi Romijn[4]

[1]University of Twente, the Netherlands
[2]RWTH Aachen University, Germany
[3]ProRail, the Netherlands
[4]Movares Nederland, the Netherlands

Home › International Journal on Software Tools for Technology Transfer › Article

DFT modeling approach for operational risk assessment of railway infrastructure

General | Special Issue: FMICS 2019/2020 | Open access | Published: 05 April 2022

Volume 24, pages 331–350, (2022) Cite this article

Download PDF

You have full access to this open access article

International Journal on Software Tools for Technology Transfer

Aims and scope →

Submit manuscript →

Norman Weik, Matthias Volk, Joost-Pieter Katoen & Nils Nießen

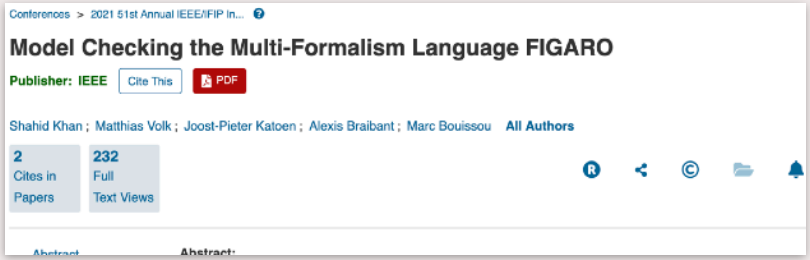
4026 Accesses 15 Citations Explore all metrics →

Use our pre-submission checklist →

Avoid common mistakes on your manuscript.

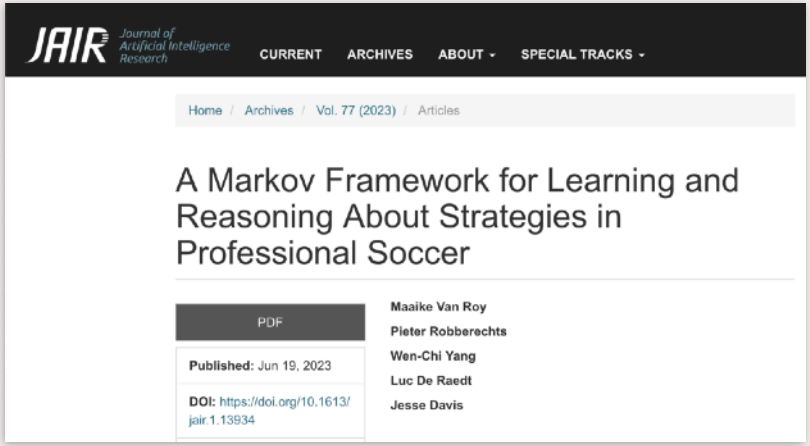
Applications of probabilistic model checking

- Network protocols
- Power management
- Reliability and Safety:
  - space
  - aerospace
  - automotive
  - railway
  - nuclear

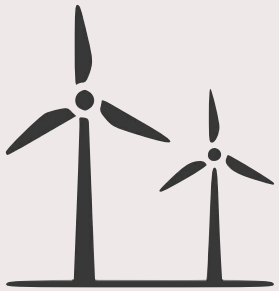


Applications of probabilistic model checking

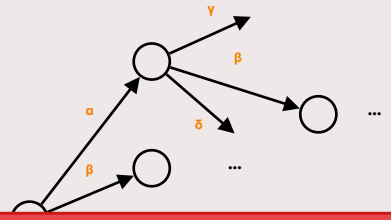
- Network protocols
- Power management
- Reliability and Safety:
  - space
  - aerospace
  - automotive
  - railway
  - nuclear
- Biology, epidemics, …,
- even soccer



Probabilistic model checking (today)



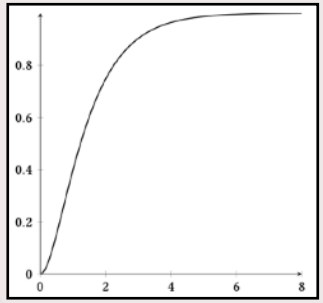
Real-world system



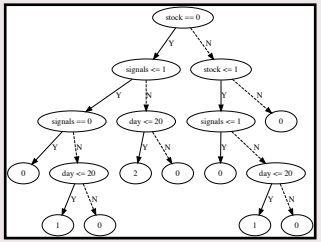
Markov model

Modern usage requires interaction and should enable custom workflows

Model Checker



Analysis Result



Diagnostic Information

Failure probability within time t

Requirement

$$\mathbf{P}_{=?}\left(\Diamond^{\leq t}\ \text{SysFail}\right)$$

Formal Specification

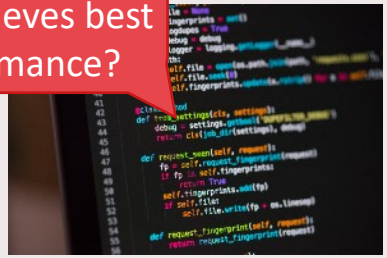
TU/e

A variety of analysis queries

How **reliable** is the system?

Component failures

What **random bias** achieves best performance?

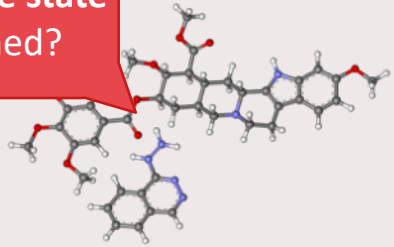


Randomized Algorithms

How to safely navigate in **unknown environments**?

Artificial Intelligence

Can a **stable state** be reached?



Biological / Chemical Systems

What is an **optimal tradeoff** between cost and punctuality?

Arrivals / Delays

What is the **winning strategy**?

Board games

TU/e

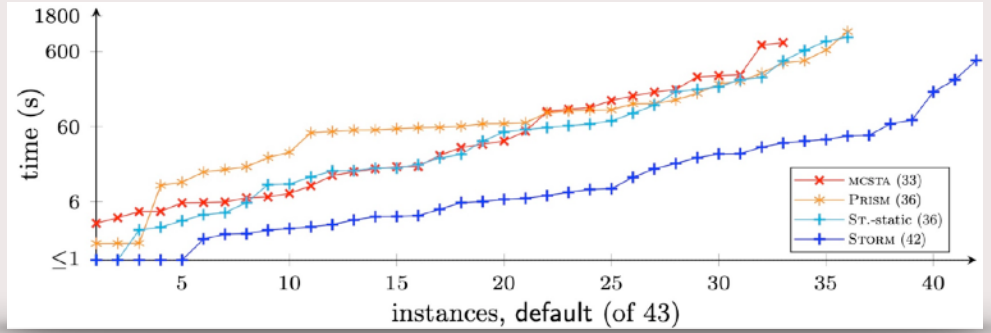
[Hensel et al, STTT 2022]

Storm model checker

www.stormchecker.org

- Modern open-source model checker
- 12 years, >10k commits, 200k LoC
- Supports various
  - types of models
  - model representations
  - analysis queries
  - …

- Efficient analysis via modular design:
  - Select from different algorithms and solver backends



[Budde et al., ISoLA 2020]

Storm model checker

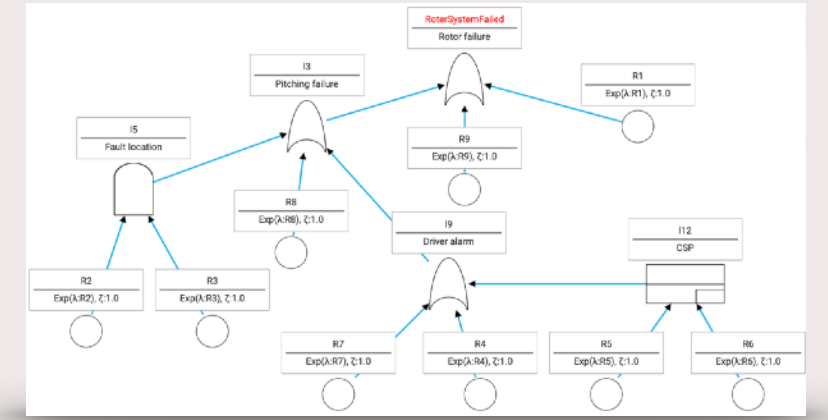
Basis for other tools

- CONVINCE
  - Robust robot planning
- CAESAR
  - Verifier for probabilistic programs
- PAYNT
  - Synthesis of probabilistic programs
- STAMINA
  - Analysis of infinite-state models
- SAFEST
  - Dynamic fault tree analysis

SAFEST
Dynamic Fault Tree Analysis

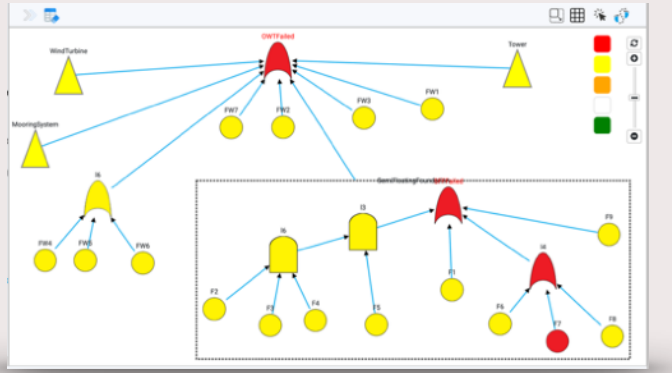
www.safest.dgbtek.com

- (Dynamic) fault trees common model in reliability analysis

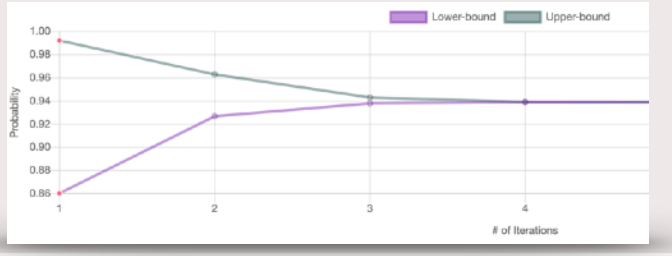


- Tool applied in industrial applications:
  - e.g. nuclear, automotive

Fault tree editor and simulator



Approximate analysis



TU/e

Storm model checker

Basis for other tools

- CONVINCE
  - Robust robot planning
- CAESAR
  - Verifier for probabilistic programs
- PAYNT
  - Synthesis of probabilistic programs
- STAMINA
  - Analysis of infinite-state models
- SAFEST
  - Dynamic fault tree analysis

Storm ecosystem

- **Stormpy**: Python API for
  - easy usage
  - dynamic interaction
  - rapid prototyping

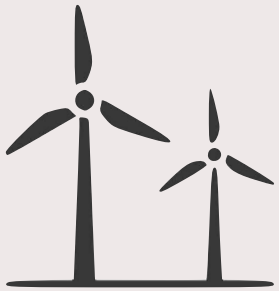
→ `pip install stormpy`

- **Stormvogel**:
  - Easy to use for novices
  - Visualizations

→ `pip install stormvogel`

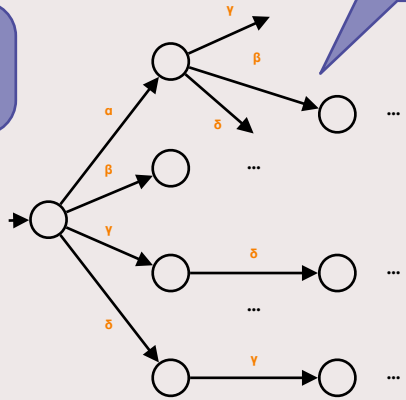
[Baier, Katoen, 2008]

Probabilistic model checking



Real-world system

How to model?



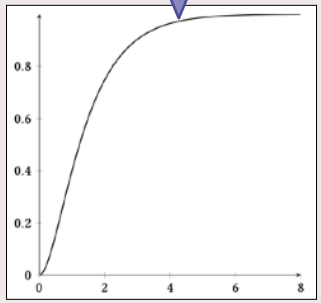
Markov model

How to incorporate uncertainty?

Storm

Model Checker

How to extract information?



Analysis Result

What to analyze?

Failure probability within time t

Requirement

$\mathbf{P}_{=?}\left(\Diamond^{\leq t}\ \text{SysFail}\right)$

Formal Specification

How to perform custom steps?

How to check?

TU/e

This tutorial

Part 1
- Background: MDP, policies
- Step 1: How to model?
- Background: properties
- Step 2: What to analyze and how to check?

Part 2
- Step 3: How to extract information?
- Background: interval models, POMDP
- Step 4: How to incorporate uncertainty?
- Step 5: How to perform custom steps?

Hands-on: execute and adapt Python code yourself

After this tutorial, you can model and analyse probabilistic systems

Outline

Part 1

- **Background: MDP, policies**
- Step 1: How to model?
- Background: properties
- Step 2: What to analyze and how to check?

Part 2

- Step 3: How to extract information?
- Background: interval models, POMDP
- Step 4: How to incorporate uncertainty?
- Step 5: How to perform custom steps?

[Katoen, LICS 2016]

Overview: Markov models

	Discrete time	Continuous time
Deterministic	**DTMC** Discrete-time Markov chain	**CTMC** Continuous-time Markov chain
Non-deterministic	**MDP** Markov decision process	**MA** Markov automaton

[Katoen, LICS 2016]

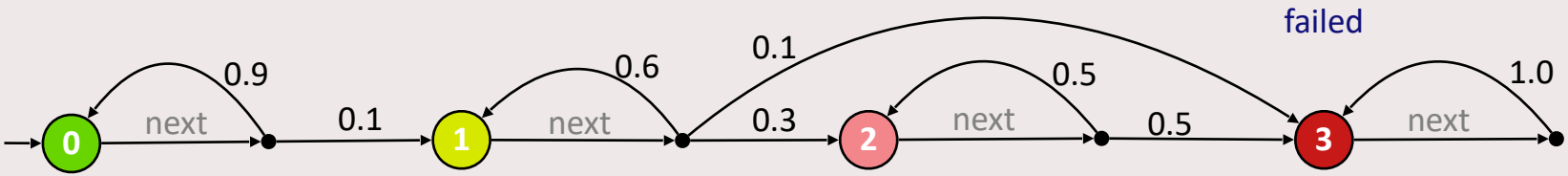
Markov Decision Process (MDP)

Support for

- discrete time
- transition probabilities and
- non-deterministic choices

Markov Decision Process (MDP)

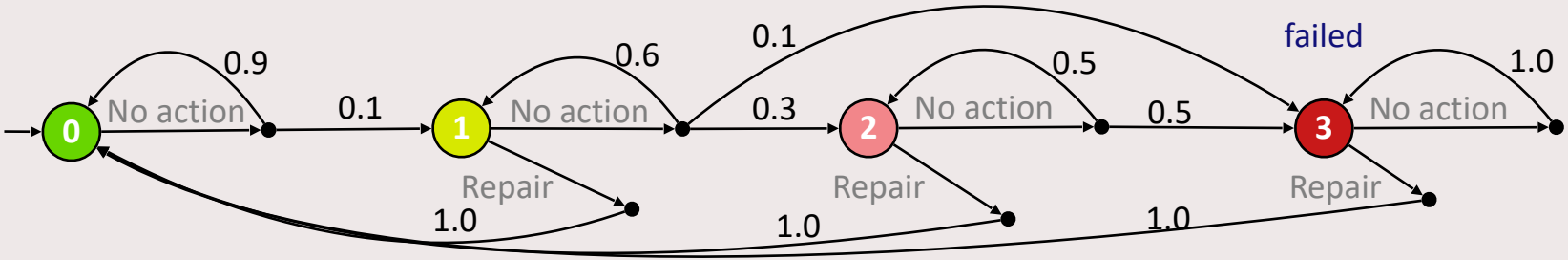
Support for
- discrete time
- transition probabilities and
- non-deterministic choices



[Katoen, LICS 2016]

Markov Decision Process (MDP)

Support for
- discrete time
- transition probabilities and
- non-deterministic choices



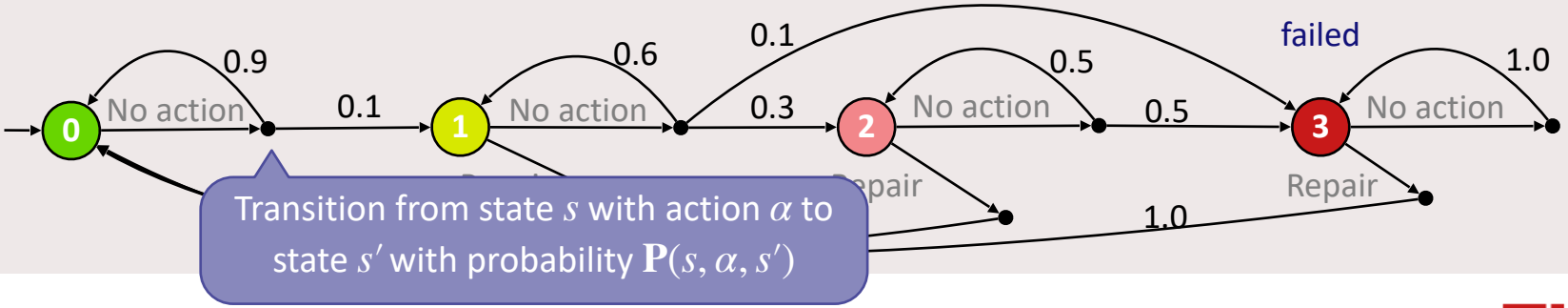
Markov Decision Process (MDP)

Definition: $\mathcal{M} = (S, s_0, Act, \mathbf{P}, AP, L)$

- Finite set of states S and initial state s_0
- Finite set of actions Act
- Partial transition function

$$\mathbf{P} : S \times Act \nrightarrow \text{Distr}(S)$$

- Write $\mathbf{P}(s, \alpha, s') = \mathbf{P}(s, \alpha)(s')$ if $\mathbf{P}(s, \alpha)$ is defined
- Write $\mathbf{P}(s, \alpha, s') = 0$, otherwise



TU/e

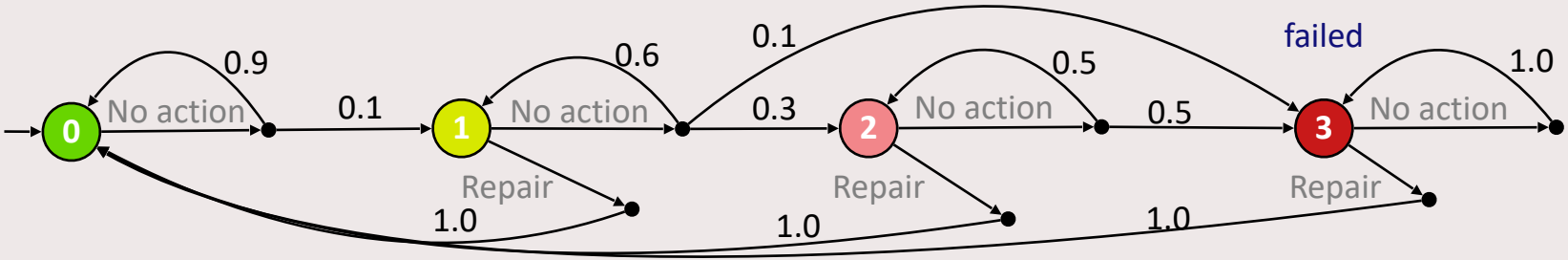
Markov Decision Process (MDP)

Definition: $\mathcal{M} = (S, s_0, Act, \mathbf{P}, AP, L)$

- Finite set of states S and initial state s_0
- Finite set of actions Act
- Partial transition function

$$\mathbf{P} : S \times Act \nrightarrow \text{Distr}(S)$$

- Finite set of atomic propositions AP
- State labeling function L: $S \to 2^{AP}$



Markov Decision Process (MDP)

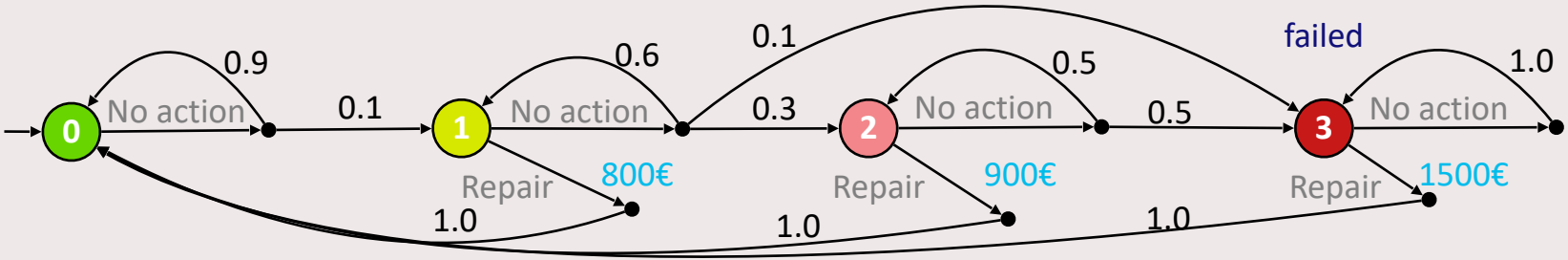
Definition: $\mathcal{M} = (S, s_0, Act, \mathbf{P}, AP, L)$

- Finite set of states S and initial state s_0
- Finite set of actions Act
- Partial transition function

$$\mathbf{P} : S \times Act \nrightarrow \text{Distr}(S)$$

- Finite set of atomic propositions AP
- State labeling function L: $S \to 2^{AP}$
- Additional: rewards (costs)

$$R: S \times Act \to \mathbb{R}_{\geq 0}$$

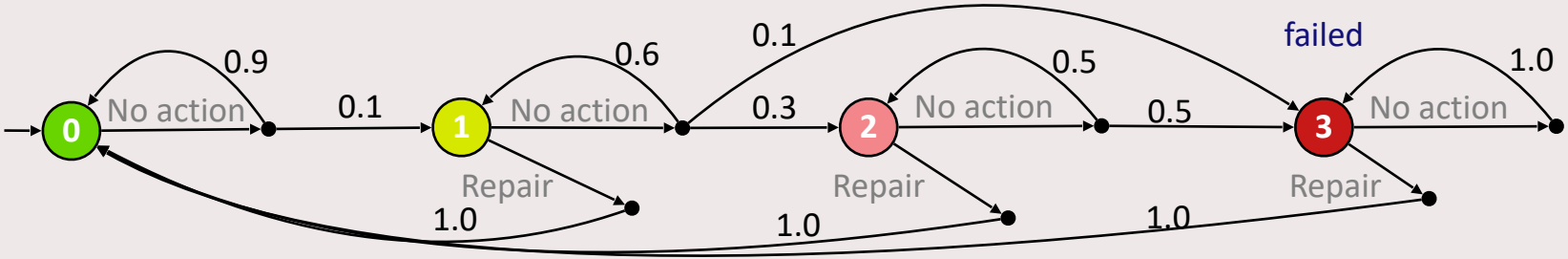


[Katoen, LICS 2016]

Markov Decision Process (MDP)

Support for
- discrete time
- transition probabilities and
- non-deterministic choices

Non-determinism can represent:
- Uncontrollable influence:
  environment, user input
- Controllable influence:
  repair strategy, control actions



Policies

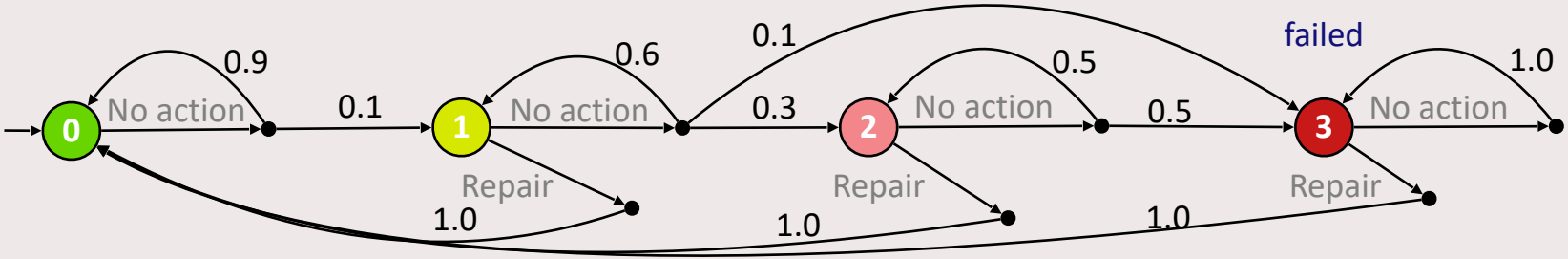
- Policy (strategy, scheduler) maps paths to actions:

$$\sigma : (S \times Act)^* \times S \to Act$$

- Memoryless policies:

$$\sigma : S \to Act$$

- Applying a policy σ yields a (deterministic) discrete-time Markov chain (DTMC) $\mathcal{M}^\sigma$



Policies

- Policy (strategy, scheduler) maps paths to actions:

$$\sigma : (S \times Act)^* \times S \to Act$$

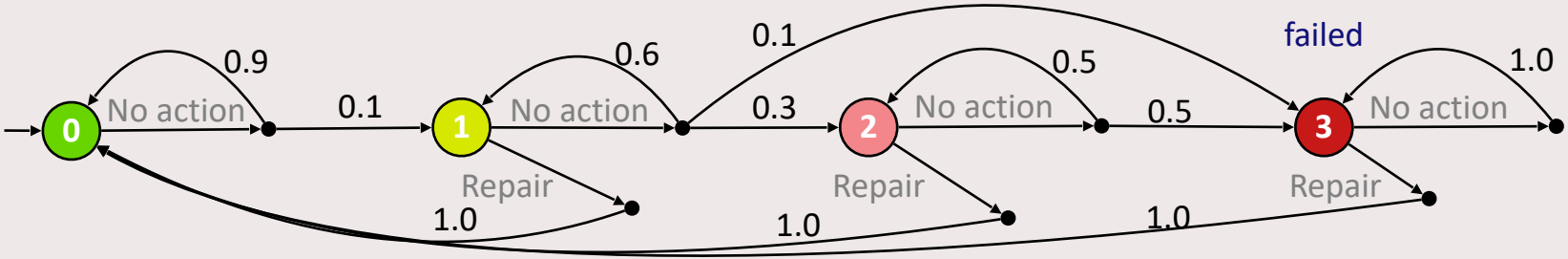
- Memoryless policies:

$$\sigma : S \to Act$$

- Applying a policy σ yields a (deterministic) discrete-time Markov chain (DTMC) $\mathcal{M}^\sigma$

Example policy

- $\pi(0) = \{\text{No action}\}$
- $\pi(1) = \{\text{No action}\}$
- $\pi(2) = \{\text{No action}\}$
- $\pi(3) = \{\text{Repair}\}$



TU/e

Policies

- Policy (strategy, scheduler) maps paths to actions:

$$\sigma : (S \times Act)^* \times S \to Act$$

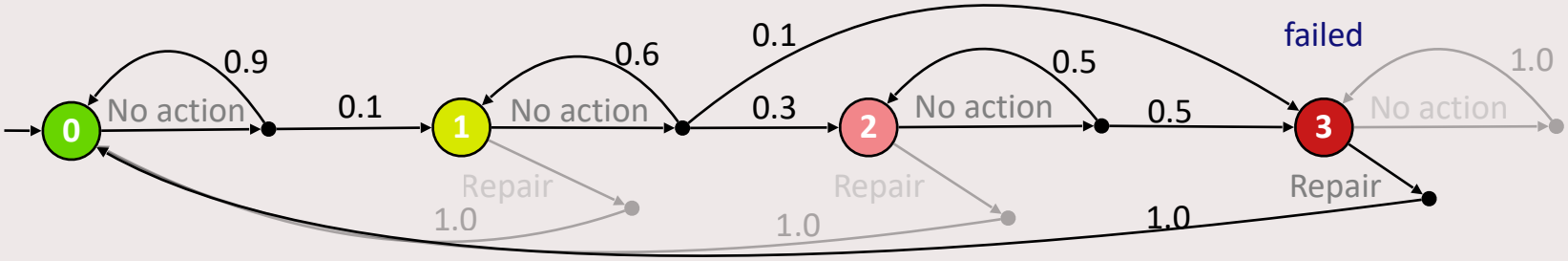
- Memoryless policies:

$$\sigma : S \to Act$$

- Applying a policy σ yields a (deterministic) discrete-time Markov chain (DTMC) $\mathcal{M}^\sigma$

Example policy

- $\pi(0) = \{\text{No action}\}$
- $\pi(1) = \{\text{No action}\}$
- $\pi(2) = \{\text{No action}\}$
- $\pi(3) = \{\text{Repair}\}$



TU/e

Outline

Part 1

- Background: MDP, policies
- **Step 1: How to model?**
- Background: properties
- Step 2: What to analyze and how to check?

Part 2

- Step 3: How to extract information?
- Background: interval models, POMDP
- Step 4: How to incorporate uncertainty?
- Step 5: How to perform custom steps?

Outline

Part 1

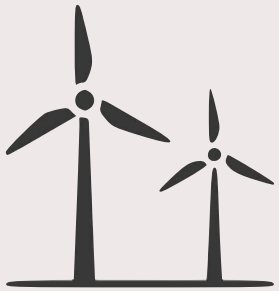
- Background: MDP, policies
- Step 1: How to model?
- **Background: properties**
- Step 2: What to analyze and how to check?

Part 2

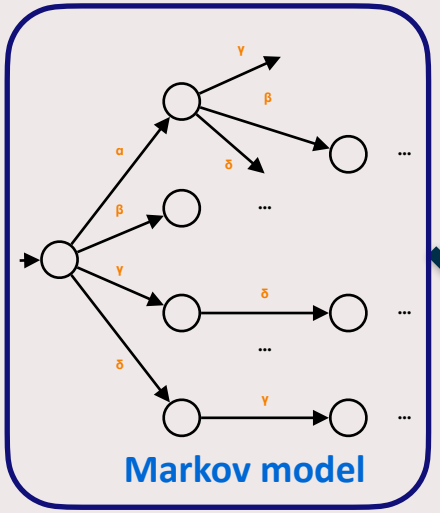
- Step 3: How to extract information?
- Background: interval models, POMDP
- Step 4: How to incorporate uncertainty?
- Step 5: How to perform custom steps?

[Baier, Katoen, 2008]

Probabilistic model checking



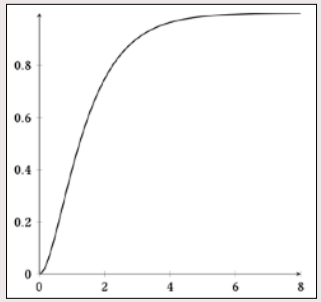
Real-world system



Markov model

Storm

Model Checker



Analysis Result

Failure probability within time t

Requirement

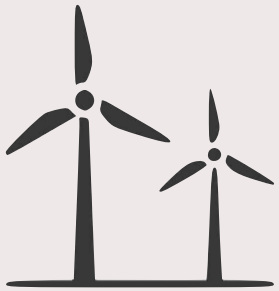
$\mathbf{P}_{=?}\left(\Diamond^{\leq t}\ \text{SysFail}\right)$

Formal Specification

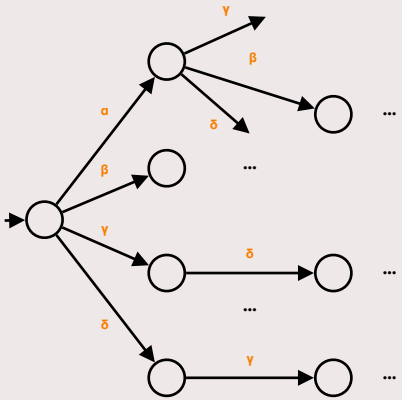
TU/e

[Baier, Katoen, 2008]

Probabilistic model checking



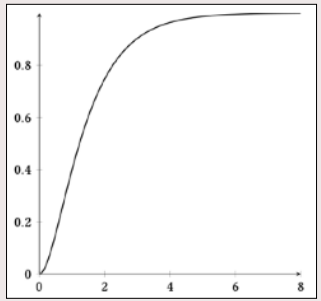
Real-world system



Markov model

Storm

Model Checker



Analysis Result

Failure probability within time t

Requirement

$$\mathbf{P}_{=?}\left(\Diamond^{\leq t}\ \text{SysFail}\right)$$

Formal Specification

TU/e

Property specification

Analysis queries in logics:

- Probabilistic computational tree logic (PCTL)
- linear temporal logic (LTL)

PCTL

PCTL state formulas

$$\Phi ::= \textbf{true} \mid a \in AP \mid \Phi_1 \wedge \Phi_2 \mid \neg\Phi \mid \mathbb{P}_J(\varphi)$$

with $\varnothing \neq J \subseteq [0,1]$

- Use $\mathbb{P}_{=?}(\varphi)$ to compute the exact probability

PCTL path formulas

$$\varphi ::= \mathbf{X}\,\Phi \mid \Phi_1\ \mathbf{U}^I\ \Phi_2$$

with $I \subseteq [0,\infty)$.

- Syntactic sugar: $F\,\Phi = \text{true}\ \mathbf{U}^{[0,\infty)}\ \Phi$

Reachability probability

- *"What is the probability to eventually reach a specific state satisfying Φ from the initial state?"*
- Depends on policy σ:

$$\mathbb{P}^{\sigma}(F\ \Phi)$$

- Maximal probability

$$\mathbb{P}_{max=?}(F\ \Phi) = \sup_{\sigma} \mathbb{P}^{\sigma}(F\ \Phi)$$

- Minimal probability

$$\mathbb{P}_{min=?}(F\ \Phi) = \inf_{\sigma} \mathbb{P}^{\sigma}(F\ \Phi)$$

Computed by **Bellman equations**:

$$x_s = \begin{cases} 1 & \text{if } s \models \Phi \\ 0 & \text{if } s \in S^{min/max}_{=0} \\ \min / \max_{\alpha \in Act(s)} \sum_{s'} \mathbf{P}(s, \alpha, s') \cdot x_{s'} & \text{otherwise} \end{cases}$$

TU/e

Rewards

"What is the maximal expected total reward until reaching a specific state satisfying Φ from the initial state?"

$$\mathbb{R}^{rew}_{max=?}(F\ \Phi)$$

for reward structure rew

And beyond:

- Reward-bounded reachability probabilities
- Conditional reachability probabilities
- Multi-objective queries
- …

Outline

Part 1

- Background: MDP, policies
- Step 1: How to model?
- Background: properties
- **Step 2: What to analyze and how to check?**

Part 2

- Step 3: How to extract information?
- Background: interval models, POMDP
- Step 4: How to incorporate uncertainty?
- Step 5: How to perform custom steps?

Outline

Part 1

- Background: MDP, policies
- Step 1: How to model?
- Background: properties
- **Step 2: What to analyze and how to check?**

Part 2

- Step 3: How to extract information?
- Background: interval models, POMDP
- Step 4: How to incorporate uncertainty?
- Step 5: How to perform custom steps?

☕ Coffee break! ☕

Outline

Part 1

- Background: MDP, policies
- Step 1: How to model?
- Background: properties
- Step 2: What to analyze and how to check?

Part 2

- **Step 3: How to extract information?**
- Background: interval models, POMDP
- Step 4: How to incorporate uncertainty?
- Step 5: How to perform custom steps?

Outline

Part 1

- Background: MDP, policies
- Step 1: How to model?
- Background: properties
- Step 2: What to analyze and how to check?

Part 2

- Step 3: How to extract information?
- **Background: interval models, POMDP**
- Step 4: How to incorporate uncertainty?
- Step 5: How to perform custom steps?

More uncertainty

Imprecise probabilities

- Probabilities allow to capture (stochastic) uncertainty
- Still need to know precise probability values

Allow imprecisions:

- Interval MDP
- Parametric MDP

Partial observability

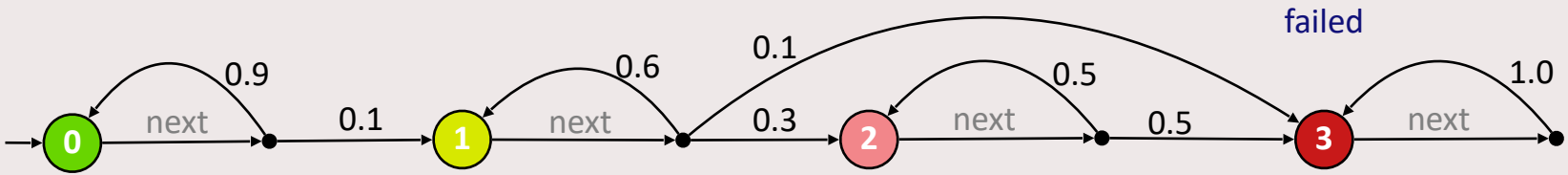
- Policy has perfect information about current state
- In reality only know about observable parts of the system and not internals

Allow partial observations:

- Partially Observable MDP (POMDP)

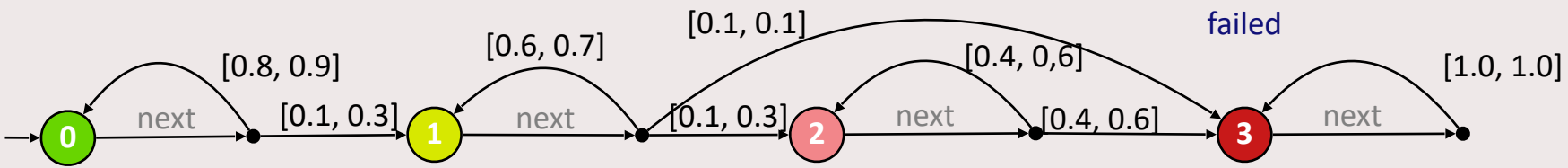
Interval MDP

- Replace precise probabilities with intervals of possible probabilities
- Think of set of MDPs, covering all point estimates which yield proper successor distributions



Interval MDP

- Replace precise probabilities with intervals of possible probabilities
- Think of set of MDPs, covering all point estimates which yield proper successor distributions



Interval MDP

- Policy needs to take uncertainty in intervals into account
- Two ways of resolving uncertainty in intervals

1. **Cooperative (angelic)**
   - Uncertainty in favor of policy
   - Maximizing policy
     → Maximizing probability

Best achievable result
in one environment

2. **Robust (demonic)**
   - Uncertainty against the policy
   - Maximizing policy
     → Minimizing probability

Best achievable result
over all environments

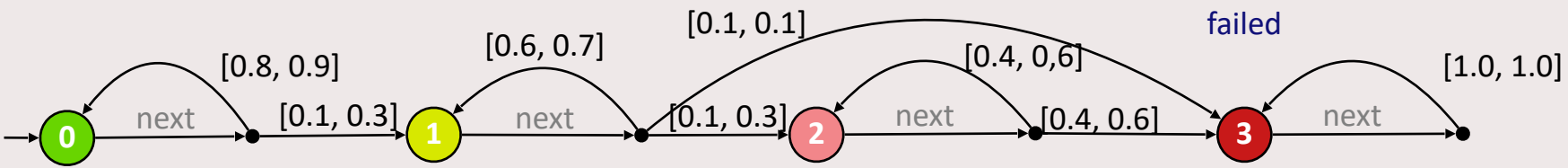
Parametric MDP

- Interval MDP bounds probabilities locally at each state
- Use parameters to capture dependencies between states

Definition: $\mathcal{M} = (S, s_0, Act, \mathbf{X}, \mathbf{P}, AP, L)$

- Finite set of parameters $\mathbf{X}$
- Transition function over rational functions

$$\mathbf{P} : S \times Act \nrightarrow S \to \mathbb{Q}(\mathbf{X})$$



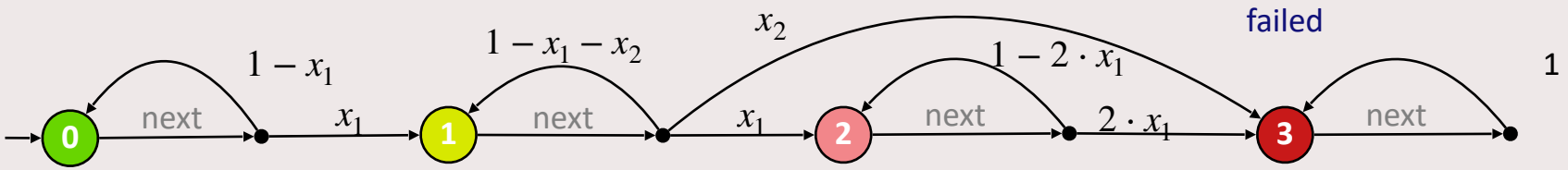
Parametric MDP

- Interval MDP bounds probabilities locally at each state
- Use parameters to capture dependencies between states

Definition: $\mathcal{M} = (S, s_0, Act, \mathbf{X}, \mathbf{P}, AP, L)$

- Finite set of parameters $\mathbf{X}$
- Transition function over rational functions

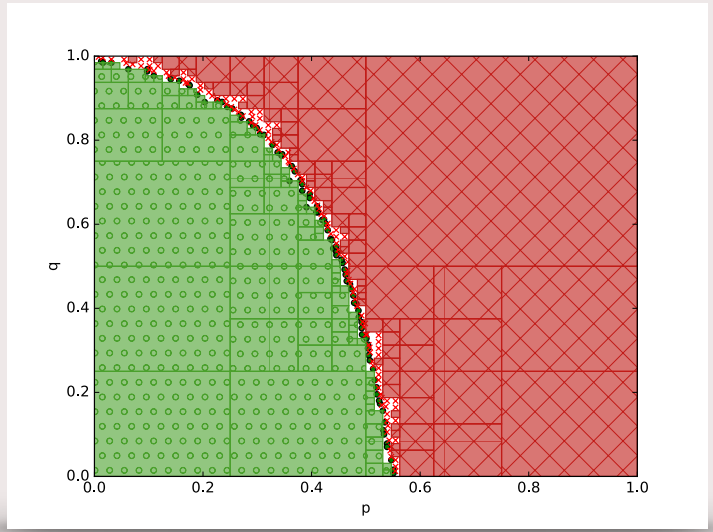
$$\mathbf{P} : S \times Act \nrightarrow S \to \mathbb{Q}(\mathbf{X})$$



Parametric MDP

Multiple analysis queries:

- Solution function:
  compute solution function for query (yields rational function)
- Feasibility:
  find parameter valuations satisfying the property
- Parameter synthesis:
  synthesis all parameter valuations satisfying/violating the property



Partially observable MDP

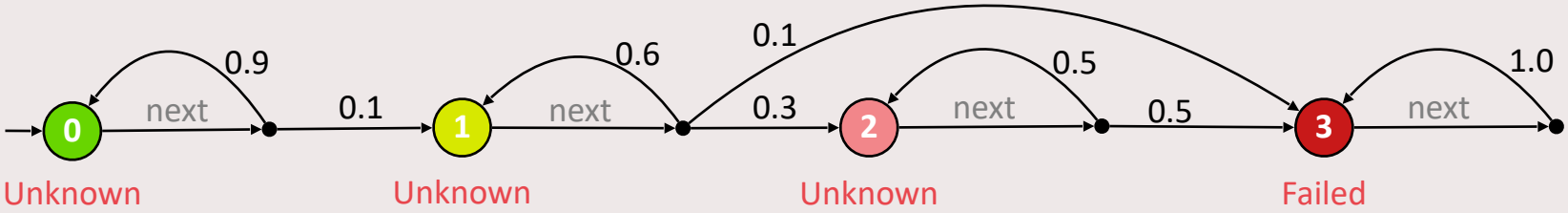
- Typically optimize over all policies
- Also policies with perfect information (might be unrealistic)

→ Only take specific observations into account

Definition: POMDP $\langle \mathcal{M}, Z, o \rangle$

- MDP $\mathcal{M}$
- Finite set of observations Z
- Observation function $o : S \to Z$ such that

$$Act(s) = Act(s') \implies o(s) = o(s')$$

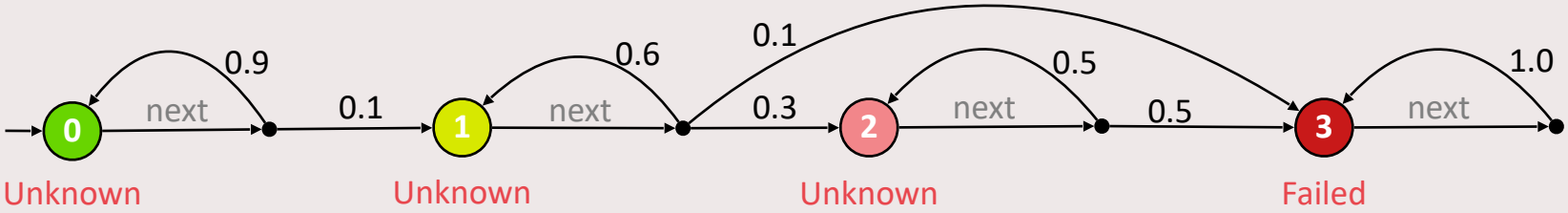


Partially observable MDP

- Policies are observation-based and history-dependent

$$\sigma : (Z \times A)^* \times Z \to A$$

- Verification of POMDP is undecidable
- Focus on obtaining bounds
- Semantics of POMDP given in terms of belief MDP



Outline

Part 1

- Background: MDP, policies
- Step 1: How to model?
- Background: properties
- Step 2: What to analyze and how to check?

Part 2

- Step 3: How to extract information?
- Background: interval models, POMDP
- **Step 4: How to incorporate uncertainty?**
- Step 5: How to perform custom steps?

Outline

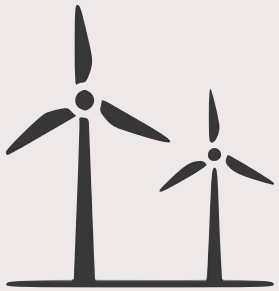
Part 1

- Background: MDP, policies
- Step 1: How to model?
- Background: properties
- Step 2: What to analyze and how to check?

Part 2

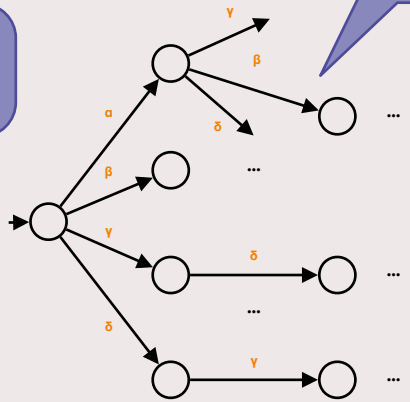
- Step 3: How to extract information?
- Background: interval models, POMDP
- Step 4: How to incorporate uncertainty?
- **Step 5: How to perform custom steps?**

Summary



Real-world system

Model in Stormvogel



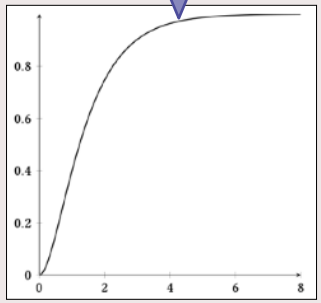
Markov model

Interval MDP, parametric MDP, POMDP

Storm

Model Checker

Compact policies



Analysis Result

Configuration of Storm

Automated analysis

Variety of properties

Failure probability within time t

Requirement

$\mathbf{P}_{=?}\left(\Diamond^{\leq t}\ \text{SysFail}\right)$

Formal Specification

TU/e

Outlook: Models

- Model representations:
  - Sparse matrix
  - Symbolic representation (decision diagrams)
- Probability representations:
  - Floating point
  - Exact (rational) numbers
  - **Intervals**
  - (Symbolic) **parameters**
- **Partial observability**

- Input languages
  - **Stormvogel** builder
  - Prism specification language
  - JANI interchange format
  - Translation from other models:
    - Dynamic fault trees
    - Petri nets

Probabilistic model checking with Storm

Modeling

- Various model types
- High-level specification

	Discrete time	Continuous time
Deterministic	**DTMC** Discrete-time Markov chain	**CTMC** Continuous-time Markov chain
Non-deterministic	**MDP** Markov decision process	**MA** Markov automaton

Zoo of properties

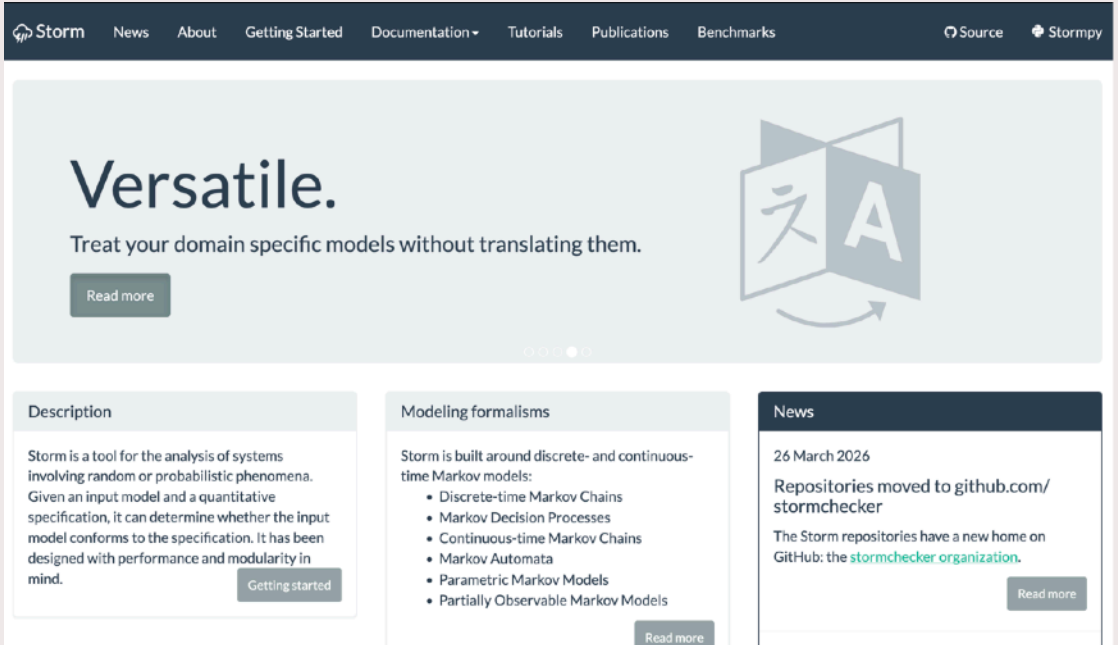
- Probability, expected rewards, optimal policy, multi-objective, …

Powerful analysis

- Automated verification
- Flexible: configure algorithms, solvers, workflow

Try it out try.stormchecker.org

www.stormchecker.org



m.volk@tue.nl

Literature

- Baier, Katoen: *"Principles of Model Checking"*
- Katoen: *"The Probabilistic Model Checking Landscape"*

- FM Tutorial paper: *"Probabilistic Model Checking Taken by Storm"* + artifact

TU/e